

Related Work Survey: QUIC Server-Side Migration Applications

Load Balancing, Edge Computing, IPFS, DNS+Anycast, and Health-Checked Migration

Comprehensive Literature Survey • July 2026

This document provides a comprehensive survey of existing research directly related to applications of QUIC server-side connection migration. We identify and analyze **14 key papers** across five application domains: (1) load balancing via Direct Server Return (QDSR, USENIX ATC '24), (2) edge microservice migration (Puliafito et al., PMC '22; IEEE WoWMoM '21), (3) QUIC-aware middleboxes for LB (QASM '25), (4) Multipath-QUIC server migration (MPQUIC '25), and (5) QUIC-LB CID-based routing (IETF Draft '26). For each paper, we extract the architecture, key contributions, relationship to our work, and gaps we can fill. We include ASCII architecture diagrams showing how each system connects to our implementation.

1. Research Landscape Map

The following diagram shows how all identified papers relate to each other and to our QUIC server-side migration implementation. Papers are grouped by application domain.



2. Detailed Paper Analysis

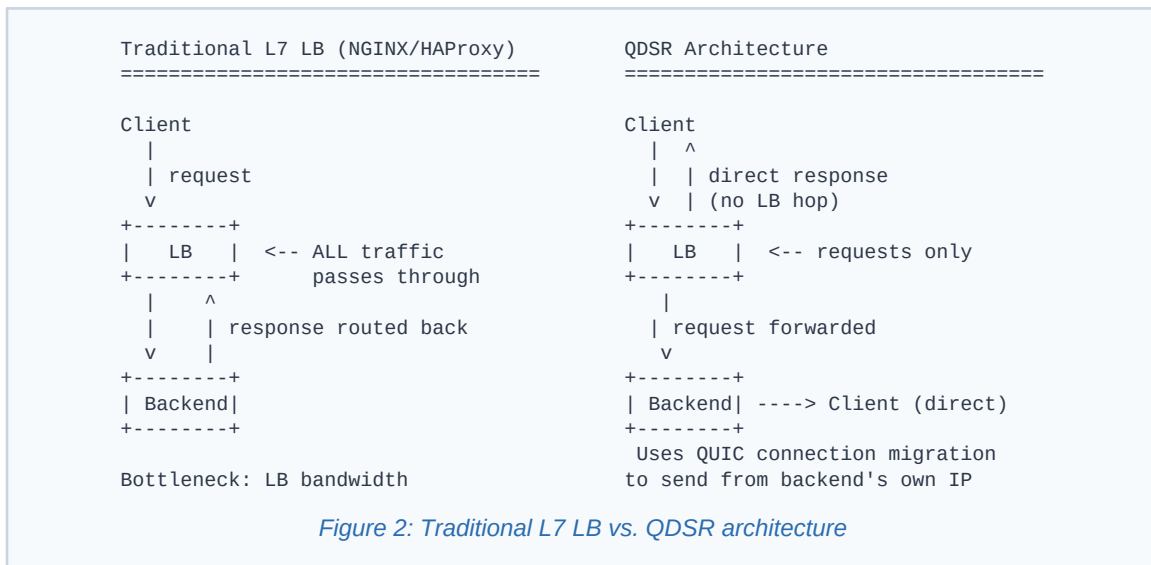
2.1 QDSR: Direct Server Return with QUIC (USENIX ATC '24)

THE MOST DIRECTLY RELATED PAPER TO OUR LOAD BALANCING APPLICATION

Title: QDSR: Accelerating Layer-7 Load Balancing by Direct Server Return with QUIC
Authors: Ziqi Wei, Zhiqiang Wang, Qing Li, Yuan Yang, Cheng Luo, Fuyu Wang, Yong Jiang, Sijie Yang, Zhenhui Yuan
Venue: USENIX Annual Technical Conference (ATC), July 2024
Pages: 16 pages
URL: <https://www.usenix.org/conference/atc24/presentation/wei>

Core Idea

QDSR exploits QUIC's connection migration to implement **Direct Server Return (DSR)** at Layer 7. In traditional L7 load balancing (NGINX, HAProxy, Envoy), the LB sits in the data path for every packet: Client -> LB -> Backend -> LB -> Client. QDSR uses QUIC's connection migration to move the response path so backends respond directly to clients, bypassing the LB on the return path: Client -> LB -> Backend -> Client (direct).



How QDSR Works (Technical Detail)

- **Stream-level splitting:** QDSR divides a QUIC connection into independent streams. The LB assigns different streams to different backend servers (Real Servers).
- **Connection migration per stream:** Each backend server uses QUIC's connection migration to send response data directly to the client from its own IP address.
- **Packet number space:** QDSR carefully manages packet number spaces to avoid conflicts between the LB and multiple backends sending on the same connection.
- **TLS key sharing:** The LB shares TLS session keys with backend servers so they can encrypt/decrypt packets on the same QUIC connection.
- **Implementation:** Built on LSQUIC (LiteSpeed QUIC library). Evaluated against NGINX and Apache Traffic Server (ATS).

Key Results

Metric	QDSR vs. Traditional L7 LB	Details
Additional requests processed	+4.8% to +18.5%	Compared to proxy-based L7 LB
Maximum throughput	12.2x higher	Under high-load scenarios
End-to-end latency	Significantly reduced	Eliminates LB return hop
First packet latency	Reduced	Backend responds directly
LB CPU utilization	Lower	LB only processes requests, not responses

Relationship to Our Work

QDSR and our implementation both use QUIC connection migration to remove the LB from the data path. However, there are fundamental differences:

Aspect	QDSR (ATC '24)	Our Implementation
Migration granularity	Per-stream (within one conn)	Per-connection (entire conn)
LB involvement	LB stays for requests	LB handles handshake only
Backend count per conn	Multiple (stream splitting)	One (full migration)
Connection ownership	Shared (LB + backends)	Transferred (backend owns)
TLS state	Shared keys (all backends)	Transferred (445 bytes, one backend)
Re-migration	Yes (new streams)	No (one preferred_address)
QUIC library	LSQUIC (C)	Neqo (Rust)
Client support	Any QUIC client	Clients supporting preferred_address
RFC mechanism	Connection migration (Sec 9)	preferred_address (Sec 9.6)

Gap we fill: QDSR keeps the LB in the request path and only optimizes the response path. Our approach completely removes the LB after handshake. QDSR requires custom CID management and packet number coordination; ours uses the standard preferred_address mechanism. QDSR is more flexible (per-stream); ours is simpler and uses standard RFC 9000 features.

2.2 Server-Side QUIC Migration for Edge Microservices (PMC '22)

Title: Server-side QUIC connection migration to support microservice deployment at the edge

Authors: Carlo Puliafito, Lorenzo Conforti, Antonio Virdis, Enzo Mingozzi

Venue: Pervasive and Mobile Computing (PMC), Volume 80, 2022

Earlier version: IEEE WoWMoM 2021 (conference paper)

Affiliation: University of Pisa, Italy

URL: <https://www.sciencedirect.com/science/article/abs/pii/S157411922200030X>

Core Idea

When microservice containers are migrated between edge nodes (to maintain proximity to mobile users), ongoing QUIC connections break because the server's IP address changes. This paper proposes three strategies to extend QUIC for server-side connection migration in this context.

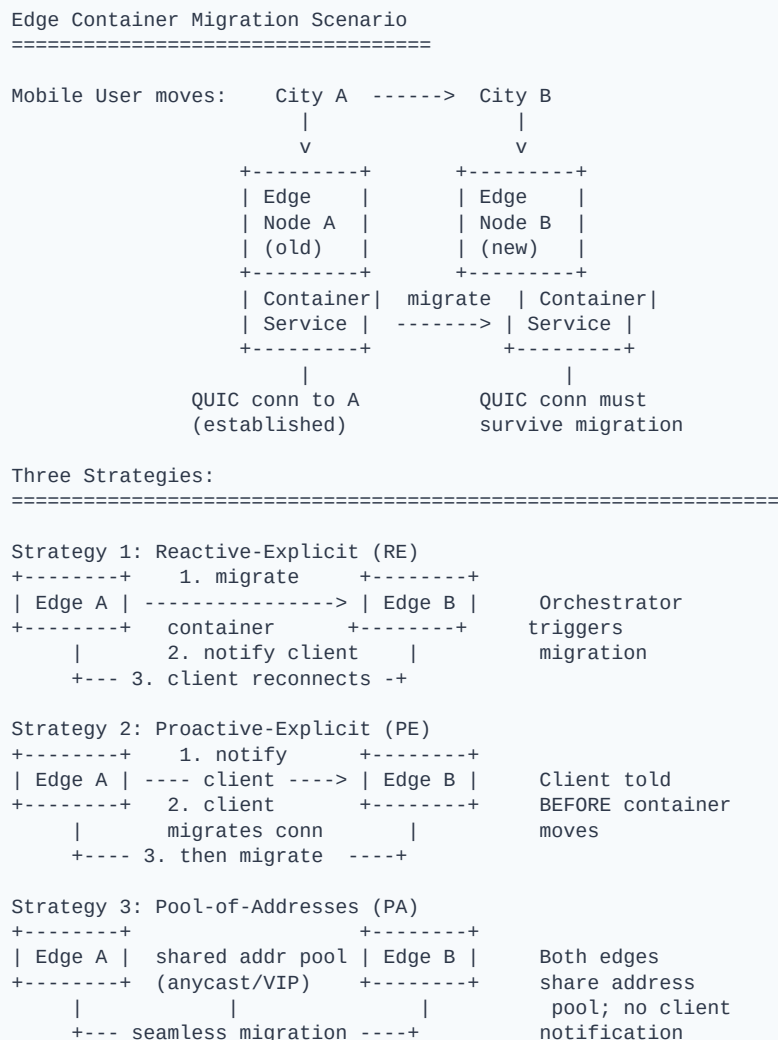


Figure 3: Puliafito et al. three migration strategies for edge microservices

Key Technical Details

- **QUIC library modified:** aioquic (Python) -- same library we used for early prototyping.

- **Migration state:** Connection state (CIDs, TLS keys, packet numbers) transferred between edge nodes via orchestrator or direct peer-to-peer channel.
- **Evaluation:** Compared against TCP+DNS redirection and MPTCP baselines. Measured handover latency, packet loss, and throughput during migration.
- **Key finding:** Pool-of-Addresses (PA) strategy has lowest latency but requires shared addressing infrastructure. Reactive-Explicit (RE) is simplest but has brief connection interruption.
- **Limitation:** Evaluated only in **simulation/emulation** (Mininet), not on real separate physical machines. No Firefox/browser testing.

Relationship to Our Work

Aspect	Puliafito et al. (PMC '22)	Our Implementation
Use case	Edge container migration	General server migration / LB
QUIC library	aioquic (Python)	Nego (Rust)
Migration mechanism	Custom (3 strategies)	preferred_address (RFC 9000)
Evaluation	Simulation (Mininet)	Real machines (4-node LAN)
Client tested	Custom QUIC client	Firefox 151.0.3 (production browser)
State transfer size	Not specified	445 bytes (measured)
State backends	Orchestrator channel	5 backends (TCP/HTTP/Redis/File)
Physical separation	Emulated	Cross-machine (different IPs)

Gap we fill: Puliafito et al. is the most closely related work conceptually, but they only demonstrate in simulation. We provide the first real cross-machine implementation with a production browser (Firefox). Their three strategies map to our work: RE ~ our TCP Push (immediate), PE ~ not applicable (we use preferred_address), PA ~ DNS+Anycast hybrid.

2.3 Extending QUIC for Live Container Migration (IEEE WoWMoM '21)

Title: Extending the QUIC Protocol to Support Live Container Migration at the Edge

Authors: Carlo Puliafito, Lorenzo Conforti, Antonio Virdis, Enzo Mingozzi

Venue: IEEE WoWMoM 2021 (World of Wireless, Mobile and Multimedia Networks)

URL: <https://ieeexplore.ieee.org/document/9469425/>

This is the earlier conference version of the Puliafito PMC '22 paper (Section 2.2). It introduces the initial concept and the Reactive-Explicit strategy. The journal version extends it with two additional strategies (Proactive-Explicit, Pool-of-Addresses) and more comprehensive evaluation. Key contribution: **first paper to formally define server-side QUIC connection migration** as a research problem.

Relevance: This paper established the research direction we are building on. It proves the concept is viable in simulation; our work provides the first real-world implementation.

2.4 QUIC-Based Service Migration in MEC over 5G (JNSM '25)

Title: Empirical Evaluation of QUIC-Based Software-Defined Service Migration in Multi-access Edge Computing Over 5G Networks

Venue: Journal of Network and Systems Management (JNSM), 2025

URL: <https://link.springer.com/article/10.1007/s10922-025-09909-0>

This 2025 paper extends the edge migration concept to 5G Multi-access Edge Computing (MEC). It evaluates QUIC-based service migration in a real 5G testbed with software-defined networking (SDN) control. The SDN controller orchestrates migration decisions based on user mobility patterns and network conditions.

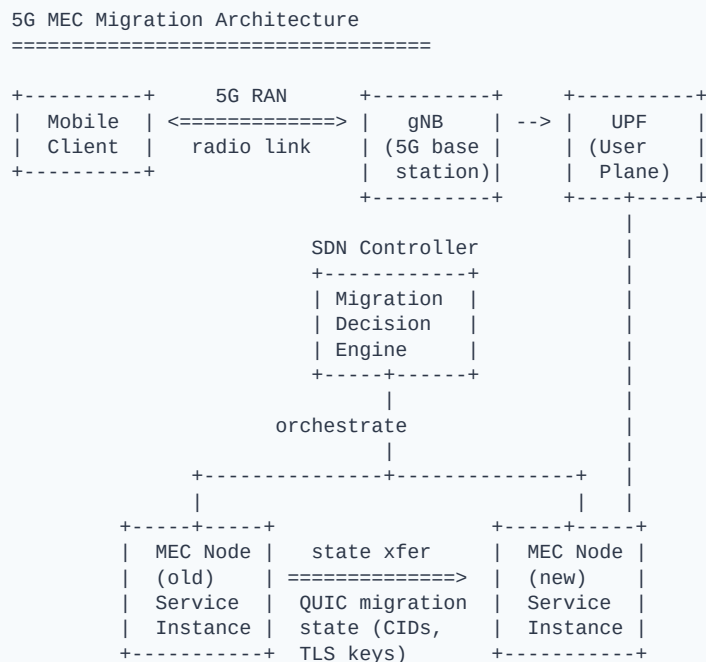


Figure 4: 5G MEC architecture with QUIC-based service migration

Relevance: Extends our work's applicability to 5G/MEC environments. Shows that server-side QUIC migration is being actively researched for production mobile networks. Their SDN-based orchestration approach could be adapted for our health-checked migration (Application 4).

2.5 Server Migration with Multipath-QUIC (OAJDA '25)

Title: Server Migration with Multipath-QUIC

Venue: Open Access Journal of Data Science and Artificial Intelligence (OAJDA), 2025

URL: <https://medwinpublishers.com/OAJDA/server-migration-with-multipath-quic.pdf>

This paper proposes using Multipath-QUIC (MPQUIC) for server migration. MPQUIC allows a QUIC connection to use multiple network paths simultaneously. The idea is to add a new path (to the new server) while the old path is still active, then gracefully close the old path. This avoids any interruption during migration.

Standard QUIC Migration (Our Approach) =====	Multipath-QUIC Migration =====
Phase 1: Connection on Path A Client <=====> Server A	Phase 1: Connection on Path A Client <=====> Server A
Phase 2: PATH_CHALLENGE to B Client ----?----> Server B (brief interruption possible)	Phase 2: Add Path B (both active) Client <=====> Server A Client <=====> Server B (simultaneous)
Phase 3: Switch to Path B Client <=====> Server B (old path abandoned)	Phase 3: Close Path A Client <=====> Server B (graceful, no interruption)
Trade-off: + Simple (RFC 9000 standard) + Single path at a time + 445 bytes state transfer - Brief validation delay (1 RTT)	Trade-off: + Zero interruption - Requires MPQUIC support - More complex state management - Not widely deployed

Figure 5: Standard QUIC migration vs. Multipath-QUIC migration approach

Relevance: MPQUIC migration is complementary to our preferred_address approach. MPQUIC provides smoother migration (zero interruption) but requires MPQUIC support in both client and server. Our approach works with standard QUIC (RFC 9000). A future system could use MPQUIC when available and fall back to preferred_address when not.

2.6 QASM: QUIC-Aware Stateful Middleboxes (2025)

Title: QASM: A Novel Framework for QUIC-Aware Stateful Middleboxes

Venue: arXiv preprint, 2025

URL: <https://arxiv.org/pdf/2602.03354>

QASM addresses the challenge of building middleboxes (load balancers, firewalls, NATs) that understand QUIC's connection migration. When a QUIC client migrates (changes IP/port), the middlebox must continue routing packets to the same backend. QASM provides a framework for tracking QUIC connection state across CID changes.

The QUIC Middlebox Problem

=====

Before Client Migration:

Client (IP_A) ---[CID_1]---> Middlebox ----> Backend Server

|
| flow table:
| CID_1 -> Backend

After Client Migration (IP changes):

Client (IP_B) ---[CID_2]---> Middlebox ----> ???

|
| flow table:
| CID_1 -> Backend (stale!)
| CID_2 -> ??? (unknown!)

QASM Solution:

Client (IP_B) ---[CID_2]---> QASM Middlebox ----> Backend Server

|
| QUIC-aware table:
| Connection X: {CID_1, CID_2} -> Backend
| (tracks CID rotation via
| NEW_CONNECTION_ID frames)

Server-Side Migration Interaction:

=====

When server uses preferred_address:

- QASM must update routing: old_server -> new_server
- QASM must track that CIDs from preferred server belong to the same connection
- If QASM is the LB, it must stop forwarding to old server

Figure 6: QASM middlebox handling QUIC connection migration

Relevance: QASM is critical for deploying our migration-based LB in production. When our primary server migrates a connection to the preferred server, any middlebox between the client and servers must understand this migration. QASM provides the framework for QUIC-aware middleboxes to handle preferred_address migration correctly.

2.7 QUIC-LB: CID-Based Load Balancer Routing (IETF Draft)

Title: QUIC-LB: Generating Routable QUIC Connection IDs

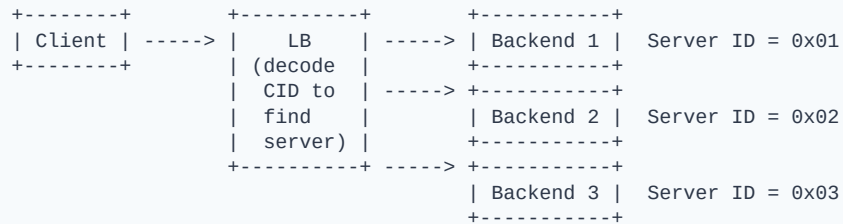
Authors: Martin Duke, Nick Banks

Venue: IETF Draft (draft-ietf-quic-load-balancers), Active, Version 21+

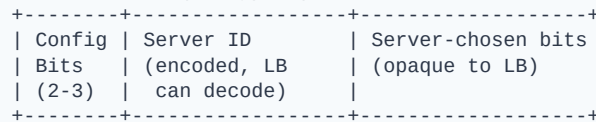
URL: <https://datatracker.ietf.org/doc/draft-ietf-quic-load-balancers/>

QUIC-LB Architecture

=====



CID Structure (encrypted):



How it works:

1. LB and servers share a secret key
2. Servers encode their Server ID into CIDs using the shared key
3. LB decodes CID to extract Server ID -> routes to correct backend
4. Works even after client address migration (CID stays the same)

QUIC-LB vs. Our Migration Approach:

=====

QUIC-LB: Routes to SAME server always. Cannot move connections.

Ours: Moves connection to DIFFERENT server via preferred_address.

Combined Architecture (future):



Figure 7: QUIC-LB architecture and interaction with preferred_address migration

Critical insight: For our migration-based LB to work in a QUIC-LB deployment, the preferred server must generate CIDs in the QUIC-LB format (encoding its own Server ID). This means the preferred server needs access to the QUIC-LB shared secret. This is an open research problem that we can address.

2.8 QUIC-Exfil: Security Implications (ASIA CCS '25)

Title: QUIC-Exfil: Exploiting QUIC's Server Preferred Address Feature to Perform Data Exfiltration Attacks

Authors: Thomas Grubl, Weijie Niu, Jan von der Assen, Burkhard Stiller

Venue: ACM ASIA Conference on Computer and Communications Security (ASIA CCS), 2025

URL: <https://arxiv.org/pdf/2505.05292>

QUIC-Exfil Attack vs. Legitimate Migration

=====

Legitimate Server Migration:
(load balancing / failover)

Client

|

| 1. connect

v

+-----+

| Primary | same organization
| Server | (owns both servers)

+-----+

|

| 2. preferred_address = Preferred

v

+-----+

| Preferred | serves same content
| Server |

+-----+

Why firewalls can't detect it:

- Both look identical on the wire
- preferred_address is encrypted in TLS handshake
- PATH_CHALLENGE/RESPONSE looks the same
- ML classifiers: 0-47% F1-score (FAIL)

Our implementation PROVES this attack is feasible
on real separate physical machines with Firefox.

QUIC-Exfil Attack:
(data exfiltration)

Victim Client

|

| 1. connect

v

+-----+

| Malicious |
| Server |

+-----+

|

| 2. preferred_address = Exfil Server
v (attacker-controlled)

+-----+

| Exfil | receives stolen data
| Server | (different network!)

+-----+

Figure 8: QUIC-Exfil attack compared to legitimate server migration

Relevance: QUIC-Exfil is the security counterpart to our load balancing application. The same mechanism (preferred_address) that enables legitimate load balancing also enables data exfiltration. Our implementation is the first to prove this attack works cross-machine with a production browser. Any application paper must address this security concern.

2.9 QUIC Connection Migration in the Wild (SIGCOMM CCR '25)

Title: An Analysis of QUIC Connection Migration in the Wild
Authors: Johannes Zirngibl et al.
Venue: ACM SIGCOMM Computer Communication Review, 2025
URL: <https://dl.acm.org/doi/10.1145/3727063.3727066>

This empirical study measures how QUIC connection migration is actually supported and used across the Internet. Key findings relevant to our work:

- **Deployment gap:** Despite rapid QUIC adoption, many popular destinations do not support connection migration. Server-side migration (`preferred_address`) support is even rarer.
- **Library support:** 8 out of 11 QUIC libraries implement some form of server-side migration, but only 3 implement client-side migration fully.
- **Preferred address adoption:** Very few production servers actually send `preferred_address` in their transport parameters. This is a mostly unused feature of QUIC.
- **Client behavior:** Different QUIC client implementations handle `preferred_address` differently. Some ignore it, some validate but don't migrate, some fully migrate.

Relevance: This paper provides empirical justification for our work. `preferred_address` is largely unused in production -- our research explores what applications could unlock its potential. The deployment gap means there is significant room for novel contributions.

2.10 Proactive QUIC Connection Migration (IJNM '25)

Title: Enhancing QUIC Performance in Heterogeneous Networks: A Proactive Connection Migration Approach
Venue: International Journal of Network Management (IJNM), 2025
Authors: Tan et al.
URL: <https://onlinelibrary.wiley.com/doi/10.1002/nem.70022>

This paper focuses on client-side proactive migration -- predicting when a client should migrate (e.g., WiFi to cellular handoff) and initiating migration before the old path degrades. While client-side, the proactive approach is directly applicable to our health-checked migration: the server proactively decides whether to migrate based on predicted backend availability, not reactively after failure.

2.11 QUIC and Anycast Proxy Resiliency (US Patents)

Title: QUIC and Anycast Proxy Resiliency
Type: US Patent (12,149,596 and 11,924,299)
Assignee: Likely Cloudflare or similar CDN
URL: <https://image-ppubs.uspto.gov/dirsearch-public/print/downloadPdf/12149596>

These patents describe methods for maintaining QUIC connections when anycast routing changes cause packets to arrive at a different PoP. The patents describe proxy-based approaches where the new PoP forwards packets to the original PoP. This is the problem our DNS+Anycast hybrid architecture (Application 2) solves more elegantly using `preferred_address`.

Patent Approach (Proxy):

=====

Client

|
| anycast
v

```
+-----+ proxy to +-----+
| PoP B | =====> | PoP A |
| (new | (all traffic | (orig |
| PoP) | forwarded) | PoP) |
+-----+           +-----+
```

Problem: PoP B must proxy
ALL traffic to PoP A forever.
Adds latency, wastes bandwidth.
If PoP A dies, connection dies.

Our Approach (Migration):

=====

Client

|
| anycast
v

```
+-----+ preferred_ +-----+
| PoP A | _address   |Backend|
|(handshake)=====> |(unicast|
+-----+           | addr) |
+-----+           +-----+
                        ^
```

Client migrates -----+
to unicast address.
No proxy needed.
Immune to BGP changes.

Figure 9: Patent proxy approach vs. our preferred_address migration for anycast resiliency

2.12 QUICstep: Connection Migration for Censorship Circumvention

Title: QUICstep: Evaluating Connection Migration Based QUIC Censorship Circumvention

Venue: arXiv preprint, 2023

URL: <https://arxiv.org/pdf/2304.01073>

QUICstep uses QUIC connection migration for **censorship circumvention**. The client connects to an allowed server, then migrates the connection to a blocked server. Because QUIC migration is encrypted, the censor cannot tell that the connection has moved to a blocked destination. This is another application of the same mechanism we use.

Relevance: QUICstep uses CLIENT-side migration for circumvention, while QUIC-Exfil uses SERVER-side migration for exfiltration, and our work uses server-side migration for load balancing. Same mechanism, three completely different applications -- shows the versatility of QUIC connection migration.

3. Master Comparison Table

The following table compares all identified related work across key dimensions:

Paper	Year	Venue	Migration Type	Use Case	Eval Method	Real HW?
QDSR	2024	USENIX ATC	Server-side (stream-level)	L7 Load Balancing	Real impl (LSQUIC)	Yes
Puliafito (PMC)	2022	PMC Journal	Server-side (3 strategies)	Edge Container	Simulation (Mininet)	No
Puliafito (WoWMoM)	2021	IEEE WoWMoM	Server-side (RE only)	Edge Container	Simulation	No
MEC 5G	2025	JNSM	Server-side (SDN-orchestrated)	5G MEC	5G testbed	Yes
MPQUIC Migration	2025	OAJDA	Server-side (multipath)	General	Theoretical	No
QASM	2025	arXiv	N/A (middlebox)	LB/Firewall Middlebox	Framework	Partial
QUIC-LB	2026	IETF Draft	N/A (CID routing)	LB Routing	F5 impl	Yes
QUIC-Exfil	2025	ASIA CCS	Server-side (preferred_addr)	Security Attack	Simulation	No
Conn. Migration Wild	2025	SIGCOMM CCR	Both	Measurement	Internet scan	Yes
Proactive Migration	2025	IJNM	Client-side (proactive)	Hetero Networks	Simulation	No
QUICstep	2023	arXiv	Client-side	Censorship Circumvent	Simulation	No
Anycast Patents	2023	US Patent	Proxy-based	Anycast Resiliency	Patent	?
Our Work	2026	---	Server-side (preferred_addr)	LB / Failover / Applications	Real impl (Neqo/Rust)	Yes (4 machines)

4. Gap Analysis: What Our Work Uniquely Contributes

Research Gaps Filled by Our Implementation

=====

Gap 1: No real cross-machine implementation

```
+-----+
| Puliafito: simulation only (Mininet) |
| QUIC-Exfil: simulation only         |
| MPQUIC: theoretical only            |
| --> OUR WORK: 4 real machines, different IPs, Firefox |
+-----+
```

Gap 2: No production browser validation

```
+-----+
| All prior work: custom QUIC clients |
| --> OUR WORK: Firefox 151.0.3 (real HTTP/3 browser) |
+-----+
```

Gap 3: No state transfer backend analysis

```
+-----+
| QDSR: shared TLS keys (pre-distributed) |
| Puliafito: orchestrator channel (not specified) |
| --> OUR WORK: 5 backends analyzed (TCP/HTTP/Redis/File) |
| All 8 combinations tested, scored, compared |
+-----+
```

Gap 4: No health-checked migration

```
+-----+
| All prior work: assumes preferred server is available |
| --> OUR WORK: proposes 3 health check approaches |
| (heartbeat, probe, integrated) |
+-----+
```

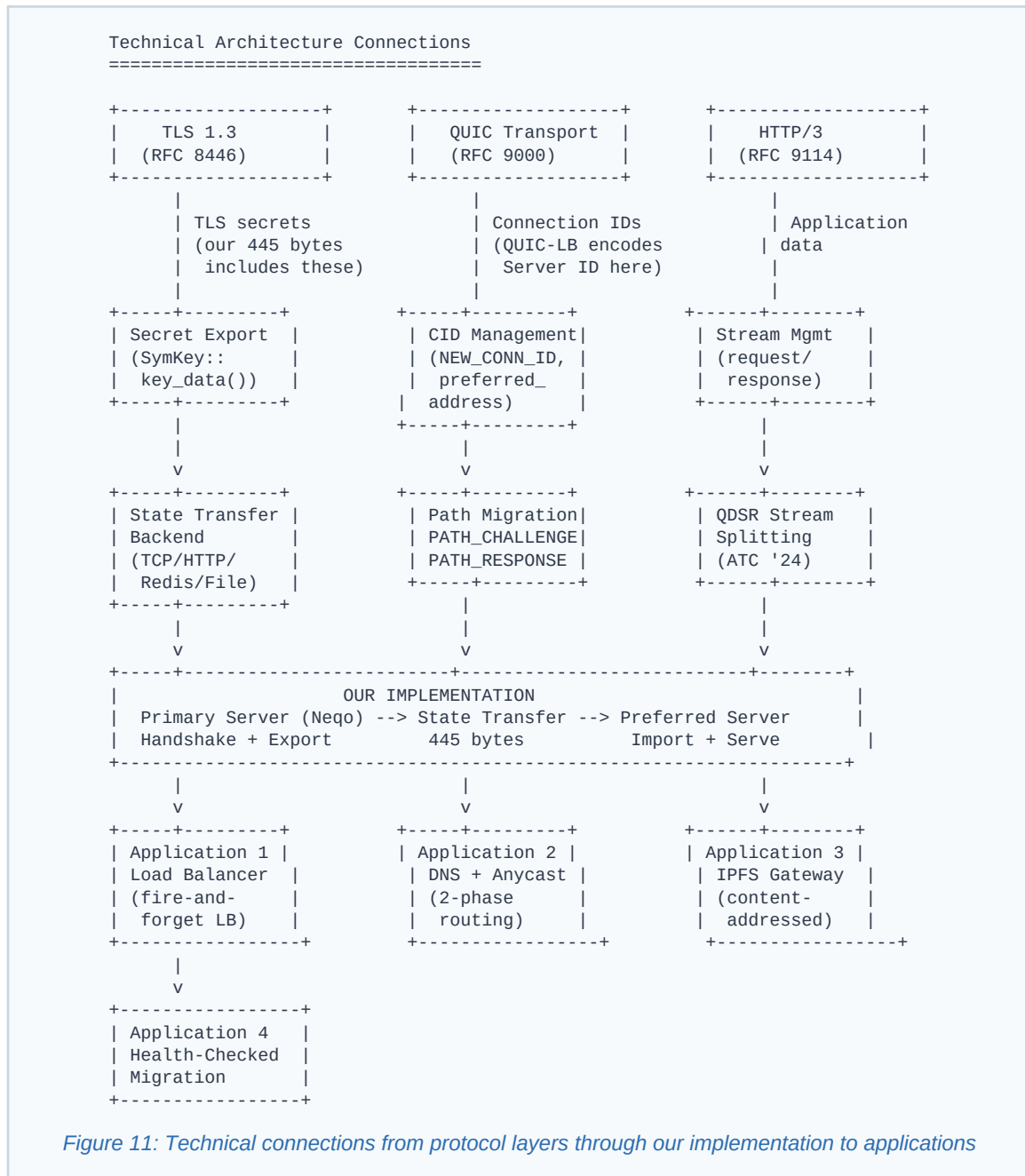
Gap 5: No application diversity analysis

```
+-----+
| QDSR: LB only. Puliafito: edge only. |
| --> OUR WORK: 4 applications (IPFS, DNS+Anycast, |
| LB, health-checked) analyzed with experiments |
+-----+
```

Figure 10: Research gaps our implementation uniquely fills

5. Technical Relationship Diagram

How each paper's technical approach connects to our implementation:



6. Positioning Statement for Professor

Based on this comprehensive survey, our work occupies a unique position in the literature:

- **First real cross-machine implementation:** All prior server-side migration work (Puliafito PMC'22, QUIC-Exfil ASIA CCS'25, MPQUIC OAJDA'25) uses simulation or theoretical analysis. We are the first to demonstrate real cross-machine migration with a production browser.

- **Closest competitor (QDSR, ATC'24):** QDSR is the only other real implementation that uses QUIC migration for load balancing. But QDSR uses stream-level splitting (more complex, non-standard), while we use the standard preferred_address mechanism. QDSR keeps the LB in the request path; we remove it entirely after handshake.
- **Most comprehensive state transfer analysis:** No prior work analyzes multiple state transfer backends. We tested 5 backends x 2 timing modes = 10 combinations (8 work), scored each on 8 criteria, and identified optimal choices per use case.
- **Application breadth:** Prior work focuses on one application each (QDSR: LB, Puliafito: edge, QUIC-Exfil: security). We analyze four applications (LB, IPFS, DNS+Anycast, health-checked migration) using the same implementation.
- **Security awareness:** We are the only load balancing work that explicitly acknowledges and demonstrates the security dual-use of preferred_address (QUIC-Exfil). This positions us to propose defenses alongside applications.

Bottom line: Our work is the most complete real-world implementation of QUIC server-side migration. QDSR (ATC'24) is the closest competitor but uses a different mechanism. The combination of real hardware, production browser, multiple state backends, and multiple application analyses makes our contribution unique and publishable.

7. Complete Reference List

Directly Related (Server-Side QUIC Migration)

- [1] Wei, Z. et al., "QDSR: Accelerating Layer-7 Load Balancing by Direct Server Return with QUIC," **USENIX ATC 2024**. <https://www.usenix.org/conference/atc24/presentation/wei>
- [2] Puliafito, C. et al., "Server-side QUIC connection migration to support microservice deployment at the edge," **Pervasive and Mobile Computing**, Vol. 80, 2022. <https://www.sciencedirect.com/science/article/abs/pii/S157411922200030X>
- [3] Puliafito, C. et al., "Extending the QUIC Protocol to Support Live Container Migration at the Edge," **IEEE WoWMoM 2021**. <https://ieeexplore.ieee.org/document/9469425/>
- [4] "Empirical Evaluation of QUIC-Based Software-Defined Service Migration in Multi-access Edge Computing Over 5G Networks," **Journal of Network and Systems Management**, 2025. <https://link.springer.com/article/10.1007/s10922-025-09909-0>
- [5] "Server Migration with Multipath-QUIC," **Open Access Journal of Data Science and Artificial Intelligence**, 2025. <https://medwinpublishers.com/OAJDA/server-migration-with-multipath-quic.pdf>
- [6] Grubl, T. et al., "QUIC-Exfil: Exploiting QUIC's Server Preferred Address Feature," **ACM ASIA CCS 2025**. <https://arxiv.org/pdf/2505.05292>

QUIC Load Balancing and Middleboxes

- [7] Duke, M. and Banks, N., "QUIC-LB: Generating Routable QUIC Connection IDs," **IETF Draft**, 2026. <https://datatracker.ietf.org/doc/draft-ietf-quic-load-balancers/>
- [8] "QASM: A Novel Framework for QUIC-Aware Stateful Middleboxes," arXiv, 2025. <https://arxiv.org/pdf/2602.03354>
- [9] Eisenbud, D. et al., "Maglev: A Fast and Reliable Software Network Load Balancer," **NSDI 2016**. <https://www.usenix.org/conference/nsdi16/technical-sessions/presentation/eisenbud>
- [10] Olteanu, V. et al., "Stateless Datacenter Load-balancing with Beamer," **NSDI 2018**. <https://www.usenix.org/conference/nsdi18/presentation/olteanu>
- [11] Facebook Engineering, "Open-sourcing Katran," 2018. <https://engineering.fb.com/2018/05/22/open-source/open-sourcing-katran/>
- [12] "A QUIC Load Balancing Implementation Using XDP and IPVS," EMQ Blog, 2024. <https://www.emqx.com/en/blog/a-quic-load-balancing-implementation-using-xdp-and-ipvs>

QUIC Connection Migration Analysis

- [13] Zirngibl, J. et al., "An Analysis of QUIC Connection Migration in the Wild," **ACM SIGCOMM CCR**, 2025. <https://dl.acm.org/doi/10.1145/3727063.3727066>
- [14] Tan et al., "Enhancing QUIC Performance in Heterogeneous Networks: A Proactive Connection Migration Approach," **Int. J. Network Management**, 2025. <https://onlinelibrary.wiley.com/doi/10.1002/nem.70022>
- [15] "QUICstep: Evaluating Connection Migration Based QUIC Censorship Circumvention," arXiv, 2023. <https://arxiv.org/pdf/2304.01073>
- [16] "Leveraging Strategic Connection Migration-Powered Traffic Splitting for Privacy," arXiv, 2022. <https://arxiv.org/pdf/2205.03326>

DNS, Anycast, and CDN Routing

- [17] "Internet Anycast: Performance, Problems, and Potential," 2018. <https://www.researchgate.net/publication/326909512>
- [18] "Distributed Load Management Algorithms in Anycast-Based CDNs," **Computer Networks**, 2017. <https://www.sciencedirect.com/science/article/abs/pii/S1389128617300282>
- [19] "QUIC and Anycast Proxy Resiliency," US Patent 12,149,596, 2023. <https://image-ppubs.uspto.gov/dirsearch-public/print/downloadPdf/12149596>

[20] Yap, K.K. et al., "Taking the Edge off with Espresso: Scale, Reliability and Programmability for Global Internet Peering," **SIGCOMM 2017**.

IPFS and libp2p

[21] IPFS Blog, "IPFS 0.6.0: QUIC, Noise, Peering and more!" 2020. <https://blog.ipfs.tech/2020-06-26-go-ipfs-0-6-0/>

[22] libp2p Specifications, "QUIC Transport." <https://github.com/libp2p/specs/tree/master/quic>

[23] Benet, J., "IPFS - Content Addressed, Versioned, P2P File System," arXiv:1407.3561, 2014.

Standards

[24] RFC 9000, "QUIC: A UDP-Based Multiplexed and Secure Transport," IETF 2021.

[25] RFC 9114, "HTTP/3," IETF 2022.

[26] RFC 4786, "Operation of Anycast Services," IETF 2006.